DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

THE UNIVERSITY OF TEXAS AT ARLINGTON

ARCHITECTURAL DESIGN SPECIFICATION

CSE 4316 / CSE 4317: SENIOR DESIGN

Spring/Summer 2026

AgriHome

GOVINDA AWASTHI

LAXMAN BHUSHAL

THUC DANG

SAGAR KHADKA

MUHAMMAD SAAD MEMON

ALEX TABLIZO

NATHAN NGUYEN TRAN

REVISION HISTORY

Revision	Date	Author(s)	Description
0.1	03.04.2026	GA, AT	Document created; initial architecture draft
1.0	03.04.2026	SK	Layered overview, figures, and PDF verification
1.1	03.04.2026	TD, MM	Raised system-level abstraction in overview sections
2.0	03.04.2026	NT, AT	Restructured to ADS format with subsystems and interfaces
3.0	07.26.2026	LB, SK	Comprehensive architectural design updates

CONTENTS

LIST OF FIGURES

LIST OF TABLES

1 INTRODUCTION

AgriHome is a smart indoor plant monitoring platform built around repeatable image capture, plant-health history, device status, and optional computer-vision analysis. This Architectural Design Specification documents the system architecture at the level needed by project evaluators, maintainers, and future implementers. The document follows common architecture-description and software-design-description practices [1, 2].

1.1 PURPOSE

The purpose of this document is to describe the high-level architecture of AgriHome, define the primary layers and subsystems, identify major interfaces, and explain how data moves through the system. The design is intentionally written at the architectural level: it identifies responsibilities and boundaries without becoming a source-code listing or detailed implementation manual.

1.2 SCOPE

The ADS covers the operator Vision Console, application services, optional processing adapters, relational and media storage, Raspberry Pi edge-device communication, and the rack-mounted XY camera capture hardware. It includes subsystem assumptions, responsibilities, interface summaries, and architectural diagrams. Out of scope are detailed UI mockups, exact database migration scripts, full API field-level contracts, training procedures for machine-learning models, and low-level hardware wiring diagrams.

1.3 SYSTEM SUMMARY

AgriHome addresses the limitations of manual plant record-keeping and disconnected image capture by providing a single monitoring console tied to physical rack capture hardware. The system combines authenticated operator workflows, repeatable plant imaging, stored image history, optional plant-health interpretation, scheduling, and edge-device management.

The current physical architecture uses a camera attached to a rack-mounted XY motion system. The camera moves horizontally along the rack and vertically to the selected plant row, then captures an individual plant image. A Raspberry Pi-class edge device coordinates local motion, camera capture, heartbeat reporting, command handling, image upload, and recovery behavior. This architecture supports repeatable plant imaging and plant-health monitoring concepts discussed in smart greenhouse, plant disease, and phenotyping literature [3, 4, 5].

1.4 DOCUMENT ORGANIZATION

Section 2 presents the project overview and describes the five primary layers. Section 3 gives a system description with a consolidated subsystem-and-data-flow view. Sections 4 through 8 document Data, Presentation, Application, Processing, and Hardware Layer subsystems respectively, each with assumptions, responsibilities, and interface tables. Section 9 lists references.

2 PROJECT OVERVIEW

The AgriHome Vision Console uses a layered architecture. Each layer has a distinct role, and layers interact through defined interfaces: browser requests to application services, device-key requests from Raspberry Pi edge devices, service calls inside the application layer, persistence operations toward data stores, optional calls to processing adapters, and hardware commands at the edge. This separation improves modularity, testability, and the ability to replace a component without rewriting unrelated areas.

The system comprises five primary layers:

1. **Presentation Layer** - user-facing experience in the browser or installable web application: authentication, dashboard, tray and plant views, mesh management, schedule editing, devices navigation, capture controls, charts, and image history.
2. **Application Layer** - server-side coordination: API entry, session and device-key verification, validation, domain orchestration, response assembly, image ingest, operator device actions, and edge-device routes.
3. **Processing Layer** - optional computer-vision and asynchronous capture helpers: plant-health adapters, tray or plant image analysis, edge post-ingest hooks, result normalization, and capture schedule runner.
4. **Data Layer** - durable storage: relational records for trays, plants, meshes, schedules, captures, prediction results, reports, edge devices, commands, and capture poses; media storage for images referenced from those records.
5. **Hardware Layer** - physical capture plane: Raspberry Pi edge host, local AgriHome agent, rack-mounted XY camera carriage, camera module, travel limits, and wiring.

2.1 PRESENTATION LAYER DESCRIPTION

The Presentation Layer is the primary interface between people and the product. It implements responsive layouts for desktop and mobile, supports installable web behavior where appropriate, and communicates with the Application Layer through authenticated requests. Users register or sign in, navigate trays and plants, inspect captures, review monitoring history, configure schedules, manage devices, and request immediate captures.

2.2 APPLICATION LAYER DESCRIPTION

The Application Layer is the central coordinator. It receives authenticated requests from the Presentation Layer, verifies human or device identity, validates payloads, delegates to domain services, coordinates storage updates, invokes optional processing adapters, and returns structured responses. Its edge routes handle Raspberry Pi registration, heartbeat, command polling, capture pose retrieval, image ingest, and status reporting. HTTP-facing behavior follows established protocol semantics [6].

2.3 PROCESSING LAYER DESCRIPTION

The Processing Layer performs compute-intensive or model-based work that is separate from core CRUD and capture storage. In AgriHome this includes optional plant-health classification, tray or layout analysis, inference normalization, post-ingest analysis after a Pi capture, and schedule-based capture command creation. When no external model is configured, the same architectural boundary can support deterministic or demonstration behavior while preserving the same response contract.

2.4 DATA LAYER DESCRIPTION

The Data Layer is the authoritative foundation for structured AgriHome state. Relational records hold plant history, captures, trays, meshes, schedules, devices, commands, poses, reports, and prediction results. Media storage holds image files referenced from that structured state. The design keeps data ownership and access control in the application services and storage layer rather than exposing direct database access to browsers or edge devices.

2.5 HARDWARE LAYER DESCRIPTION

The Hardware Layer is the physical capture plane. Raspberry Pi devices run the AgriHome edge agent, coordinate camera and motion hardware, report online status, claim commands, capture individual plant images, and upload image data and metadata. The hardware layer does not perform autonomous plant treatment or plant-care actuation; its primary role is repeatable monitoring through image capture.

2.6 LAYERED ARCHITECTURE FIGURE

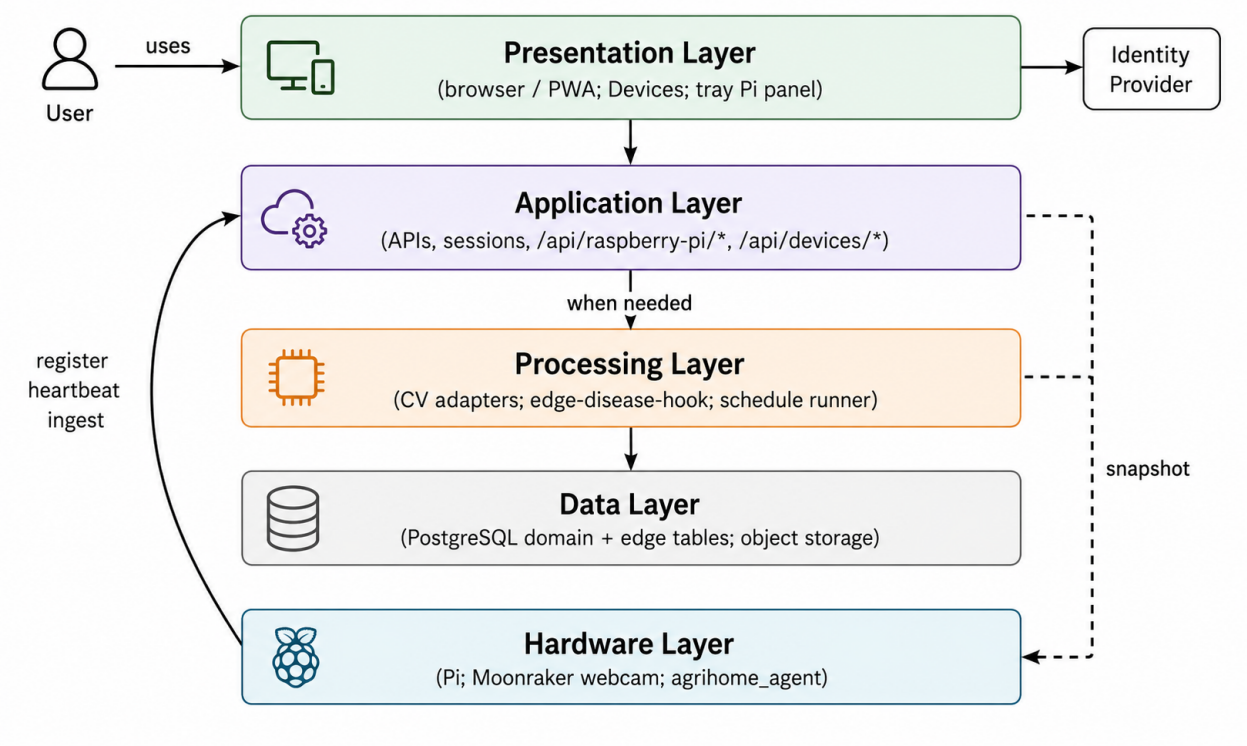

Figure 1: Five-layer architecture and the major communication paths

2.7 SYSTEM WORKFLOW OVERVIEW

A typical workflow begins when a signed-in operator opens a dashboard, tray, plant, device, or schedule view. The Presentation Layer requests data from the Application Layer, which authorizes the caller and reads the Data Layer. For Take Picture or scheduled capture, the Application Layer creates or dispatches a command to the Raspberry Pi edge device. The edge agent moves the rack-mounted XY camera carriage to the requested target, captures the image, uploads image metadata and bytes, and reports completion. Optional Processing Layer adapters analyze the image and return normalized results. The updated state flows back to the Presentation Layer for display.

3 SYSTEM DESCRIPTION

This section provides a consolidated subsystem definition and data-flow narrative for the AgriHome Vision Console and rack-mounted capture system. Figure 2 shows the physical capture hardware and application/data flow for the current rack-mounted XY camera design. Figure 3 maps major building blocks to the five layers introduced in Section 2.

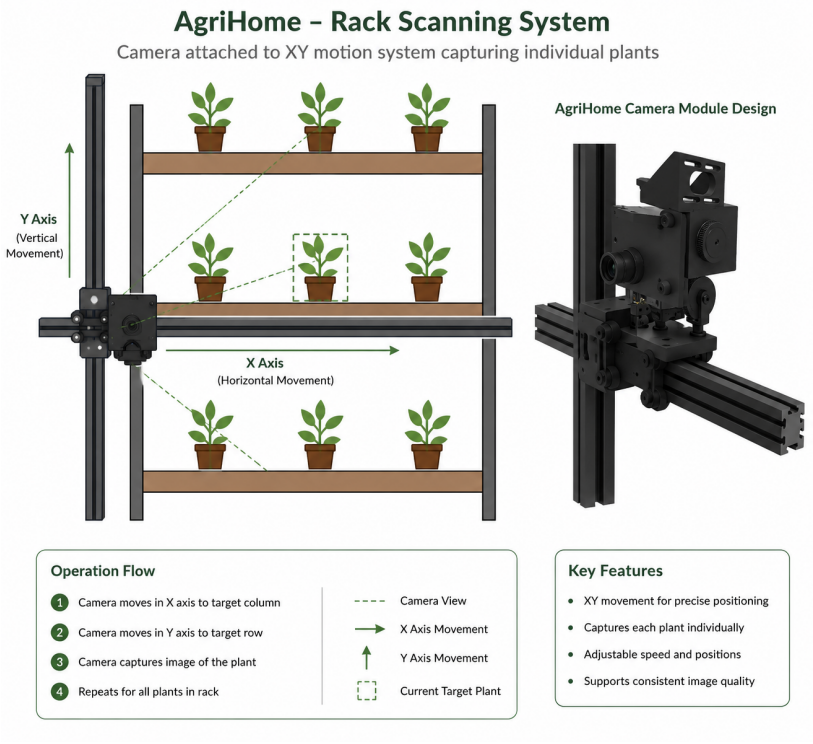

Figure 2: AgriHome Plant Capture and Monitoring System Overview

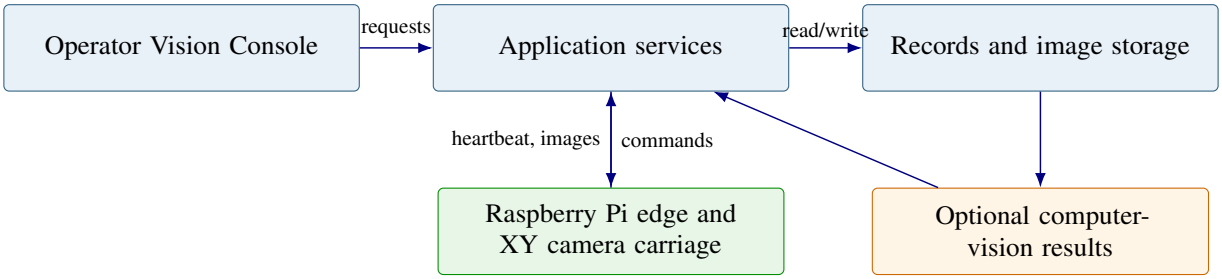

Figure 3: Subsystem definitions and primary data flow for operator actions, edge captures, storage, and optional analysis.

3.1 PRESENTATION LAYER COMPOSITION

The Presentation Layer is organized around workspaces that match operator mental models. Authentication establishes identity and ties the browser to a server session. The dashboard summarizes status and recent activity. The tray console shows imagery, plant rows, and monitoring history. The plant detail view focuses on one plant. Mesh views group trays or racks. The schedule editor defines automatic capture policy. The Devices view lists provisioned edge devices. The rack capture panel supports immediate Take Picture requests.

3.2 APPLICATION LAYER COMPOSITION

The Application Layer verifies sessions or device credentials, validates payloads, applies ownership rules, and maps caller intent to domain operations. Domain orchestration implements plant, tray, mesh, schedule, monitoring, edge registration, heartbeat, command, image ingest, and operator device workflows. Security decisions are designed around resource-specific access and explicit trust boundaries consistent with zero-trust architecture principles [7].

3.3 PROCESSING LAYER COMPOSITION

The Processing Layer exposes inference contracts and asynchronous helpers. Species or leaf adapters may classify a single-plant image. Tray vision adapters may estimate layout information when tray-level imagery is available. The edge disease hook optionally analyzes captures after ingest. The capture schedule runner evaluates due schedules and enqueues commands. Result normalization keeps labels, confidence values, geometry, and model metadata consistent before persistence.

3.4 DATA LAYER COMPOSITION

The Data Layer splits transactional concerns from bulk media. Write coordination sequences inserts and updates so relational records and file metadata remain aligned. Integrity constraints guard keys, ownership, and required relationships. Query execution handles reads and writes. Relational records store trays, plants, meshes, schedules, monitoring events, captures, predictions, reports, devices, commands, and poses. Media storage holds images.

3.5 HARDWARE LAYER COMPOSITION

The Hardware Layer is the bench capture plane: a Raspberry Pi edge host, local AgriHome agent, camera module, rack-mounted XY carriage, and physical plant rack. Agents register and heartbeat, operators trigger Take Picture, and schedules drive capture commands. The camera travels along X-axis and Y-axis paths to capture individual plants rather than relying on a single wide contextual image.

3.6 END-TO-END DATA FLOW SUMMARY

The data flow begins when a user interacts with the Presentation Layer or when an edge device heartbeats. Requests pass to the Application Layer, which authorizes and validates them. Ordinary reads query the Data Layer. Capture requests create commands for the Hardware Layer. The edge agent moves the camera carriage, captures images, and uploads them. Optional vision actions call the Processing Layer, then commit structured outputs through the Data Layer. Responses propagate back to the Presentation Layer or edge device for display, retry, or further capture steps.

4 DATA LAYER SUBSYSTEMS

The Data Layer provides durable storage for all AgriHome domain state, edge state, and media references. It is structured into coordinated write handling, integrity checks, query execution, relational persistence, edge-domain persistence, and image storage.

4.1 WRITE COORDINATION SUBSYSTEM

The write coordination subsystem sequences operations that must remain consistent across several records or across database records and file metadata. It receives structured write intents from application services and orders them so partial failures can be detected and reported.

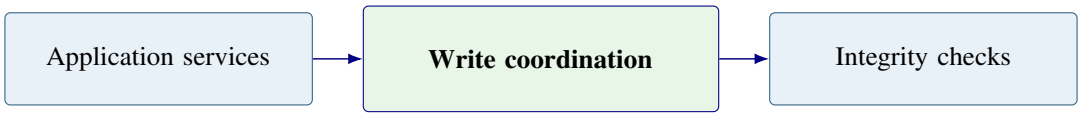

Figure 4: Write coordination subsystem: accepts write intents and forwards validated batches.

4.1.1 ASSUMPTIONS

- The Application Layer supplies writes in transactional units appropriate to each use case.
- Concurrent writers may exist, so isolation expectations are defined per operation.
- Image storage availability may differ from database availability; coordination must surface failures clearly.

4.1.2 RESPONSIBILITIES

- Accept batched write intents from domain services.
- Order dependent operations such as capture metadata, image references, and plant-status updates.
- Expose success or failure to the Application Layer for user-visible error handling.

4.1.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
D-W1	Application services to write coordination	Domain write DTOs	Ordered write plan
D-W2	Write coordination to integrity checks	Planned writes	Validation requests

Table 1: Write coordination subsystem interfaces

4.2 INTEGRITY AND CONSTRAINT ENFORCEMENT SUBSYSTEM

This subsystem enforces schema rules, ownership rules, referential integrity, and business invariants before data reaches the query executor. It combines database constraints with application-level checks that are not expressible as simple column rules.

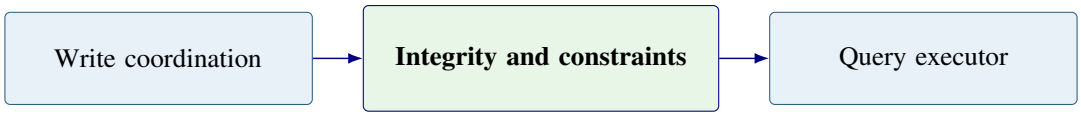

Figure 5: Integrity subsystem: validates operations before execution.

4.2.1 ASSUMPTIONS

- Invalid data may arrive from bugs, stale clients, or malicious requests.
- Constraint definitions evolve with schema migrations and implementation review.

4.2.2 RESPONSIBILITIES

- Reject writes that violate keys, ownership, required fields, or state-transition rules.
- Preserve consistency among trays, plants, captures, schedules, and edge-device records.
- Return actionable validation results to the application services.

4.2.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
D-I1	Write coordination to integrity checks	Planned writes	Validation status
D-I2	Integrity checks to query executor	Valid operations	Executable query requests

Table 2: Integrity and constraint enforcement subsystem interfaces

4.3 QUERY AND TRANSACTION EXECUTION SUBSYSTEM

The query executor is the Data Layer gateway that issues reads and writes against the relational store and coordinates binary I/O references. It provides a stable persistence interface to the Application Layer while hiding low-level query details.

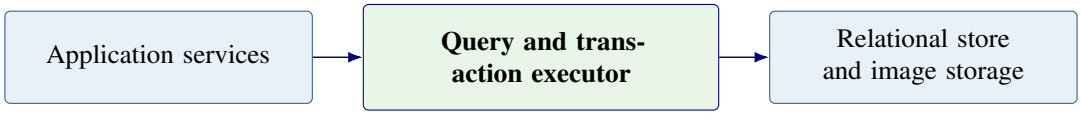

Figure 6: Query executor: issues relational queries and coordinates media references.

4.3.1 ASSUMPTIONS

- The relational database is the authoritative source for structured state.
- Large image binaries are stored outside relational rows and referenced by metadata.

4.3.2 RESPONSIBILITIES

- Execute reads for dashboards, trays, plants, captures, devices, schedules, and reports.
- Execute transactional updates after validation succeeds.
- Return normalized data structures to application services.

4.3.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
D-Q1	Application services to query executor	Read or write request	Domain records
D-Q2	Query executor to storage	SQL and media reference operations	Stored or retrieved data

Table 3: Query and transaction execution subsystem interfaces

4.4 RELATIONAL DOMAIN DATABASE SUBSYSTEM

The relational database subsystem stores authoritative domain entities such as trays, plants, meshes, schedules, camera captures, prediction results, reports, device records, commands, capture poses, and status timestamps.

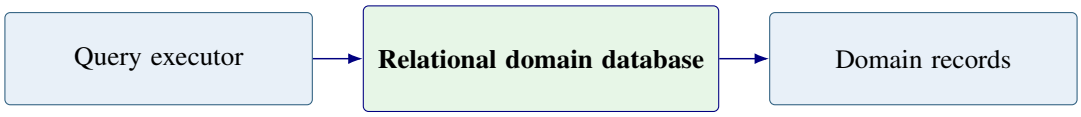

Figure 7: Relational domain database subsystem with plant, capture, schedule, and edge records.

4.4.1 ASSUMPTIONS

- The schema supports both operator workflows and edge-device workflows.
- Migration discipline is required when tables or relationships change.

4.4.2 RESPONSIBILITIES

- Persist structured AgriHome domain records.
- Maintain relationships among trays, plants, captures, schedules, devices, and results.
- Support indexed lookup for dashboard, history, and device-management views.

4.4.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
D-DB1	Query executor to relational database	Queries and mutations	Rows and transaction results
D-DB2	Relational database to application layer	Stored domain state	Structured domain objects

Table 4: Relational domain database subsystem interfaces

4.5 EDGE DOMAIN PERSISTENCE SUBSYSTEM

Edge domain persistence stores Raspberry Pi registration, heartbeat, command, capture-pose, and capture-result records. These records let the Application Layer distinguish stale devices, pending commands, completed captures, and failed capture attempts.

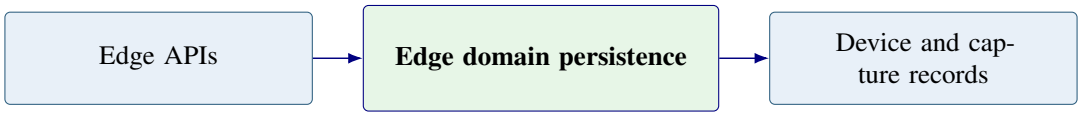

Figure 8: Edge domain persistence subsystem for device, command, pose, and capture state.

4.5.1 ASSUMPTIONS

- Edge devices may disconnect or reconnect without warning.
- Commands may remain pending until a device heartbeat claims them.
- Capture records require both device context and plant or tray context.

4.5.2 RESPONSIBILITIES

- Store device registration and heartbeat status.
- Track pending, claimed, completed, and failed capture commands.
- Associate image captures with device identity and plant or tray targets.

4.5.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
D-E1	Edge API to edge persistence	Registration, heartbeat, command status	Stored device state
D-E2	Operator devices API to edge persistence	Command requests and device updates	Command and status records

Table 5: Edge domain persistence subsystem interfaces

4.6 IMAGE AND MEDIA STORAGE SUBSYSTEM

The image and media storage subsystem retains captured images and upload files while the relational database stores references, metadata, plant context, and analysis results. This separation keeps large binaries from bloating structured domain tables.

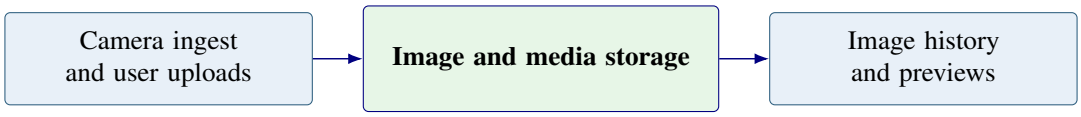

Figure 9: Image and media storage subsystem linked to capture and plant records.

4.6.1 ASSUMPTIONS

- Captured images may become large as capture frequency and plant count grow.
- The storage path must remain private unless a controlled display URL is issued.

4.6.2 RESPONSIBILITIES

- Store captured plant images and operator uploads.
- Return stable storage references for relational metadata.
- Support cleanup, retention, and backup policies for project handoff.

4.6.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
D-M1	Application services to media storage	Image bytes and metadata	Storage reference
D-M2	Media storage to presentation layer	Authorized image request	Displayable image resource

Table 6: Image and media storage subsystem interfaces

5 PRESENTATION LAYER SUBSYSTEMS

The Presentation Layer provides the operator-facing Vision Console. Its subsystems match the user workflows visible in the SRS: authentication, dashboard monitoring, tray and plant views, mesh organization, schedule editing, device management, and rack capture controls.

5.1 AUTHENTICATION AND SESSION UI SUBSYSTEM

The authentication UI collects sign-in or registration data and starts an authorized browser session. It is responsible for presenting identity-related workflows clearly without exposing secret values.

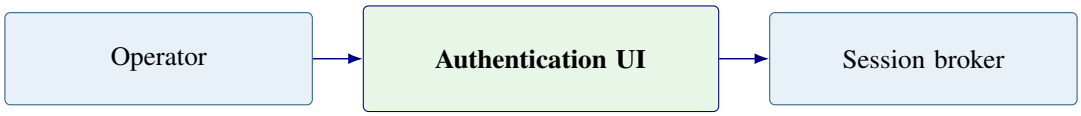

Figure 10: Authentication UI subsystem: user actions become authenticated API calls.

5.1.1 ASSUMPTIONS

- Human operators access the system through a browser or installable web application.
- Credential verification is performed by the Application Layer or selected identity service.

5.1.2 RESPONSIBILITIES

- Collect sign-in, registration, and sign-out actions.
- Display authentication errors without leaking sensitive information.
- Redirect authenticated users to the appropriate monitoring workspace.

5.1.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
P-A1	Operator to authentication UI	Credentials and navigation intent	Authentication request
P-A2	Authentication UI to session broker	Auth form payload	Session result

Table 7: Authentication and session UI subsystem interfaces

5.2 DASHBOARD AND OVERVIEW SUBSYSTEM

The dashboard subsystem summarizes plant status, recent captures, stale devices, schedule status, and important alerts. It gives the operator a fast entry point before drilling into a specific tray or plant.

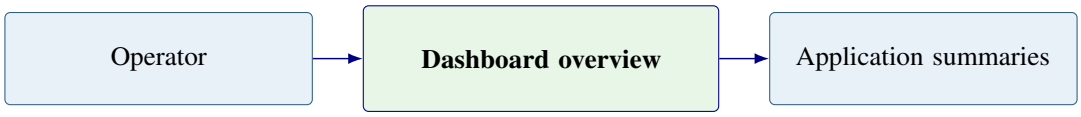

Figure 11: Dashboard subsystem requests summary aggregates from application services.

5.2.1 ASSUMPTIONS

- Dashboard data is derived from stored plant, capture, device, and schedule records.
- Summary views must remain readable on desktop and mobile screen sizes.

5.2.2 RESPONSIBILITIES

- Display plant-health summaries and recent capture activity.
- Highlight offline devices, failed captures, and other operator alerts.
- Route users to trays, plants, captures, schedules, and devices.

5.2.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
P-D1	Dashboard to application API	Summary request	Dashboard data
P-D2	Dashboard to operator	Visual status cards and tables	Navigation decision

Table 8: Dashboard and overview subsystem interfaces

5.3 TRAY MONITORING CONSOLE SUBSYSTEM

The tray monitoring console presents a tray-level view of plants, captures, history, and optional analysis results. It supports the operator mental model of monitoring one rack shelf or tray at a time.

Figure 12: Tray console subsystem: drives tray reads and optional analysis requests.

5.3.1 ASSUMPTIONS

- Each tray may contain multiple plants and multiple capture locations.
- Recent image quality and capture history are useful for diagnosing plant health.

5.3.2 RESPONSIBILITIES

- Display tray metadata, plant list, images, and health history.
- Support operator inspection workflows for individual plants.
- Expose capture and analysis actions only when the user is authorized.

5.3.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
P-T1	Tray console to application API	Tray ID and filters	Tray, plant, and capture data
P-T2	Tray console to operator	Images, status, and history	Inspection action

Table 9: Tray monitoring console subsystem interfaces

5.4 PLANT DETAIL AND HEALTH HISTORY SUBSYSTEM

The plant detail subsystem focuses on one plant record and its historical imagery, observations, predictions, and status labels. It is the primary place for reviewing changes over time.

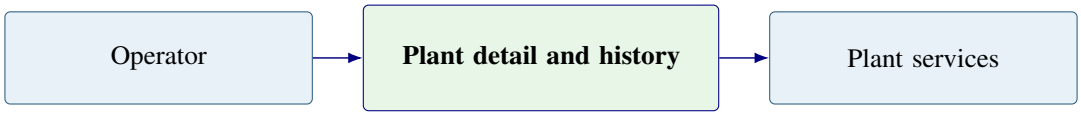

Figure 13: Plant detail subsystem: CRUD and reporting views for one plant.

5.4.1 ASSUMPTIONS

- A plant may have many associated captures and optional prediction results.
- Operators need chronological context to evaluate whether plant status is improving or worsening.

5.4.2 RESPONSIBILITIES

- Show plant identity, tray placement, and capture history.
- Present optional model confidence and limitation text when available.
- Support authorized edits to plant metadata and notes.

5.4.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
P-P1	Plant view to application API	Plant ID	Plant record and history
P-P2	Plant view to operator	Timeline and health details	Edit or review action

Table 10: Plant detail and health history subsystem interfaces

5.5 MESH AND TOPOLOGY SUBSYSTEM

The mesh and topology subsystem groups trays or plants into a higher-level monitored structure. It preserves the original ADS concept while keeping rack-based monitoring as the physical capture model.

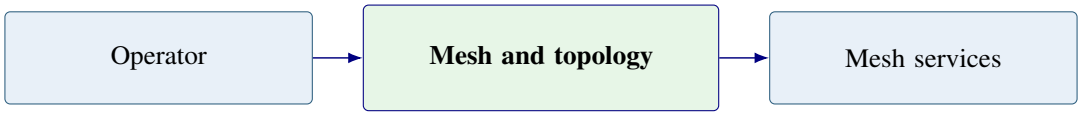

Figure 14: Mesh subsystem interacts with tray aggregates and membership records.

5.5.1 ASSUMPTIONS

- Some users may organize multiple trays, shelves, or racks as one logical collection.
- Mesh membership is an organizational feature and not a physical actuator path.

5.5.2 RESPONSIBILITIES

- Display mesh membership and aggregate status.
- Support creation and editing of logical tray groupings.
- Pass topology requests to application services.

5.5.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
P-M1	Mesh UI to application API	Mesh or tray IDs	Membership data
P-M2	Mesh UI to operator	Grouped tray status	Organization action

Table 11: Mesh and topology subsystem interfaces

5.6 SCHEDULE MANAGEMENT SUBSYSTEM

The schedule management subsystem lets authorized operators define when automatic captures should occur and which plants, trays, or capture poses should be targeted.

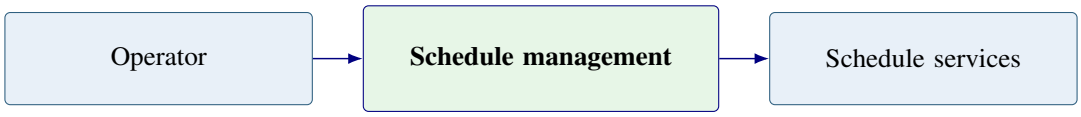

Figure 15: Schedule subsystem persists capture policies through application APIs.

5.6.1 ASSUMPTIONS

- Capture schedules must not command unsafe or impossible motion positions.
- Schedules may be disabled, edited, or reviewed before demonstration.

5.6.2 RESPONSIBILITIES

- Create, display, edit, and disable capture schedules.
- Associate schedules with trays, plants, or capture-pose sequences.
- Expose schedule status and next-run information.

5.6.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
P-S1	Schedule UI to application API	Schedule policy	Stored schedule
P-S2	Schedule UI to operator	Schedule status	Review or edit action

Table 12: Schedule management subsystem interfaces

5.7 DEVICES UI SUBSYSTEM

The Devices UI lists registered edge devices, heartbeat status, linked trays, last capture status, and available operator actions. It allows the team to demonstrate device-management behavior without exposing low-level secrets.

Figure 16: Devices UI subsystem: operator fleet view over edge-device APIs.

5.7.1 ASSUMPTIONS

- Devices can be online, stale, offline, revoked, or pending configuration.
- Operators should see status without seeing hashed or raw API keys.

5.7.2 RESPONSIBILITIES

- Display registered Raspberry Pi edge devices and heartbeat status.
- Support linking devices to trays and viewing last capture results.
- Expose safe administrative actions such as revoke, relink, or request capture.

5.7.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
P-DEV1	Devices UI to operator devices API	Device list or action request	Device status and result
P-DEV2	Devices UI to operator	Fleet status view	Device-management action

Table 13: Devices UI subsystem interfaces

5.8 RACK CAPTURE CONTROL PANEL AND TAKE PICTURE SUBSYSTEM

The rack capture control panel provides the operator-facing Take Picture workflow. The panel sends a selected tray, plant, or capture-pose target to the Application Layer, which then coordinates the Raspberry Pi edge device and XY camera carriage.

Figure 17: Rack capture panel: Take Picture requests pass through the devices API.

5.8.1 ASSUMPTIONS

- The camera is mounted to an XY carriage attached to the rack or shelf.
- Immediate captures depend on device availability, travel limits, and camera readiness.

5.8.2 RESPONSIBILITIES

- Let an authorized operator request an immediate plant capture.
- Display command state such as pending, claimed, completed, or failed.
- Show the returned image record when the capture workflow completes.

5.8.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
P-C1	Capture panel to devices API	Target tray, plant, or pose	Command record
P-C2	Capture panel to operator	Capture status and image result	Review action

Table 14: Rack capture control panel and Take Picture subsystem interfaces

6 APPLICATION LAYER SUBSYSTEMS

The Application Layer is the trusted coordination tier between user-facing screens, edge devices, processing adapters, and durable storage. Its subsystems handle routing, identity, validation, domain orchestration, response assembly, and edge-device APIs.

6.1 API ENTRY AND ROUTING SUBSYSTEM

The API entry subsystem is the first server-side hop for browser, operator, and edge-device requests. It maps routes to the correct application service and preserves stable contracts for clients.

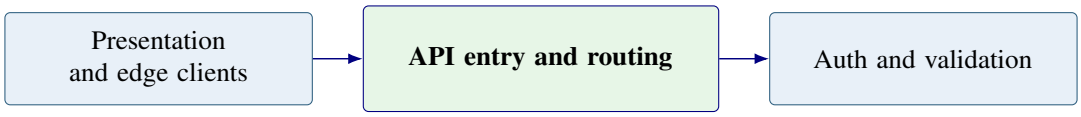

Figure 18: API entry subsystem: first server hop after presentation or edge clients.

6.1.1 ASSUMPTIONS

- External clients communicate through documented HTTP endpoints or equivalent application routes.
- Routing must keep operator actions separate from device-key actions.

6.1.2 RESPONSIBILITIES

- Receive requests from browser workflows and Raspberry Pi edge devices.
- Route requests to session, validation, domain, or device handlers.
- Apply consistent request logging and error boundaries.

6.1.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
A-R1	Client to API entry	HTTP request	Routed application call
A-R2	API entry to validation	Route context and payload	Validation request

Table 15: API entry and routing subsystem interfaces

6.2 SESSION AND AUTHENTICATION BROKER SUBSYSTEM

The session broker establishes the trusted user or device context for downstream work. Human users authenticate through session-aware flows; Raspberry Pi devices authenticate with device-specific credentials.

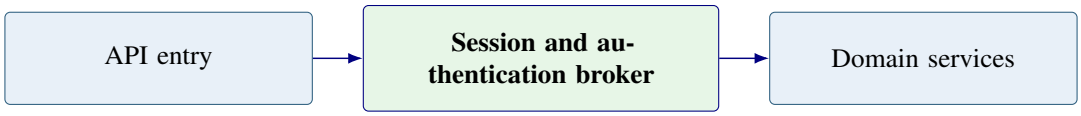

Figure 19: Session broker: establishes trusted user or device context.

6.2.1 ASSUMPTIONS

- User sessions and device credentials are separate trust paths.
- Revoked or unknown device credentials must not reach state-changing services.

6.2.2 RESPONSIBILITIES

- Verify human sessions for browser workflows.
- Verify device credentials for edge-device registration, heartbeat, command, and ingest routes.
- Attach caller context for authorization checks.

6.2.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
A-A1	API entry to session broker	Cookies, tokens, or device key	Caller context
A-A2	Session broker to domain services	Authorized context	Permission-aware request

Table 16: Session and authentication broker subsystem interfaces

6.3 INPUT VALIDATION SUBSYSTEM

The validation subsystem rejects malformed payloads, unsafe target positions, unexpected files, and invalid command parameters before side effects occur.

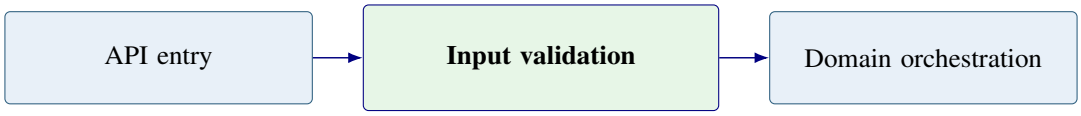

Figure 20: Validation subsystem: rejects bad input early.

6.3.1 ASSUMPTIONS

- Requests may come from stale clients, manual testing, or compromised callers.
- Server-side validation is required even when client-side forms also validate input.

6.3.2 RESPONSIBILITIES

- Validate IDs, route parameters, file metadata, JSON payloads, and capture targets.
- Enforce size, type, and required-field expectations.
- Return structured errors for display or device retry behavior.

6.3.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
A-V1	API entry to validation	Raw request data	Validated payload
A-V2	Validation to response handler	Validation failure	Structured error

Table 17: Input validation subsystem interfaces

6.4 DOMAIN ORCHESTRATION SUBSYSTEM

Domain orchestration is the central business-logic layer. It coordinates plant, tray, mesh, schedule, device, capture, image-ingest, and optional analysis use cases.

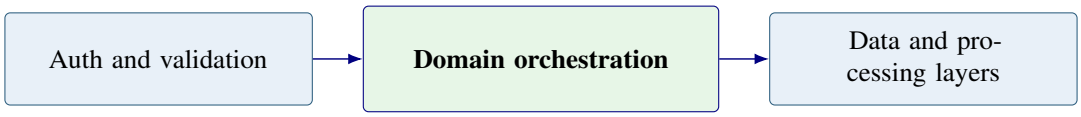

Figure 21: Domain orchestration subsystem: central business logic and integration point.

6.4.1 ASSUMPTIONS

- Most user-visible workflows require more than one persistence or processing step.
- Optional processing failures should not corrupt core monitoring records.

6.4.2 RESPONSIBILITIES

- Implement tray, plant, schedule, capture, device, and report workflows.
- Coordinate writes across relational storage and media storage.
- Invoke processing adapters when image analysis is configured.

6.4.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
A-O1	Validated request to domain orchestration	Use-case command	Domain result
A-O2	Domain orchestration to data layer	Read or write operation	Stored state

Table 18: Domain orchestration subsystem interfaces

6.5 QUERY AND MUTATION FACADE SUBSYSTEM

The query and mutation facade provides a consistent application-facing shape for reads and writes. It shields presentation code from internal service boundaries and allows implementation details to change behind stable operations.

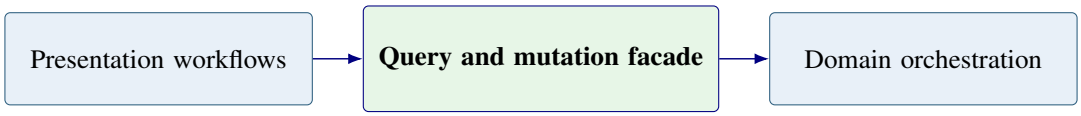

Figure 22: Query and mutation facade: thin mapping to domain services.

6.5.1 ASSUMPTIONS

- The browser should not call persistence modules directly.
- Facade functions may be implemented as HTTP route handlers, RPC handlers, or other server functions.

6.5.2 RESPONSIBILITIES

- Map client requests to domain-oriented operations.
- Return stable response shapes to the Presentation Layer.
- Hide internal data layout and service boundaries from the UI.

6.5.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
A-F1	Presentation layer to facade	Query or mutation request	Stable response shape
A-F2	Facade to domain orchestration	Domain command	Domain result

Table 19: Query and mutation facade subsystem interfaces

6.6 RESPONSE AND ERROR HANDLING SUBSYSTEM

The response handler maps successful domain results and failure states to operator-readable responses, device-readable status codes, and consistent error bodies.

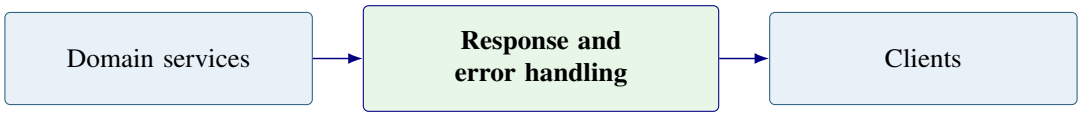

Figure 23: Response handler: maps internal results to client responses.

6.6.1 ASSUMPTIONS

- Errors may originate in validation, authorization, storage, processing, network, or hardware stages.
- Device clients need machine-readable status while human users need clear messages.

6.6.2 RESPONSIBILITIES

- Format success responses for UI and edge clients.
- Map internal failures to structured errors without leaking secrets.
- Preserve enough status information for troubleshooting and retry logic.

6.6.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
A-E1	Domain services to response handler	Result or error	Response body
A-E2	Response handler to clients	HTTP status and body	Displayed or consumed result

Table 20: Response and error handling subsystem interfaces

6.7 RASPBERRY PI EDGE API SUBSYSTEM

The Raspberry Pi edge API handles device registration, heartbeat reporting, command polling, image ingest, pose retrieval, and capture status updates from trusted edge devices.

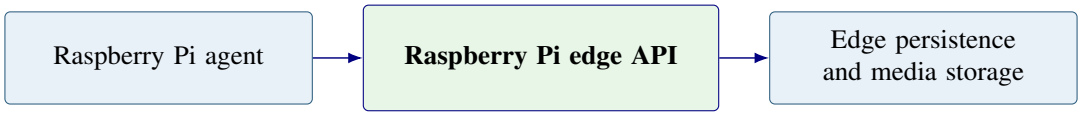

Figure 24: Raspberry Pi edge API subsystem: device-key routes into domain persistence.

6.7.1 ASSUMPTIONS

- Edge devices may be offline for long periods and must resynchronize safely.
- Image ingest must be authenticated and associated with the correct device and target.

6.7.2 RESPONSIBILITIES

- Register or update edge devices and heartbeat timestamps.
- Provide pending movement or capture commands to devices.
- Accept image uploads and capture-result metadata.

6.7.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
A-P1	Agent to edge API	Register, heartbeat, command poll, ingest	Device status and commands
A-P2	Edge API to data layer	Device and capture records	Stored edge state

Table 21: Raspberry Pi edge API subsystem interfaces

6.8 OPERATOR DEVICES API SUBSYSTEM

The operator devices API supports user-initiated device management, device linking, capture requests, credential rotation, and command-status inspection.

Figure 25: Operator devices API subsystem: Take Picture and device administration.

6.8.1 ASSUMPTIONS

- Only authorized operators may issue capture commands or change device links.
- A capture command may remain pending until the device polls for work.

6.8.2 RESPONSIBILITIES

- Create capture commands for selected plants, trays, or poses.
- Return device status, last heartbeat, and command history.
- Support safe administrative actions such as relink, revoke, and rotate credentials.

6.8.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
A-D1	Devices UI to operator devices API	Management or capture request	Command or status response
A-D2	Operator devices API to edge persistence	Device action	Updated edge records

Table 22: Operator devices API subsystem interfaces

7 PROCESSING LAYER SUBSYSTEMS

The Processing Layer contains optional computer-vision adapters and asynchronous capture helpers. It turns image input and scheduled policy into normalized results or pending edge capture work.

7.1 SPECIES AND LEAF INFERENCE ADAPTER SUBSYSTEM

The species and leaf inference adapter provides a controlled boundary around single-image classification or feature extraction. It is optional and can be replaced without changing storage or UI contracts.

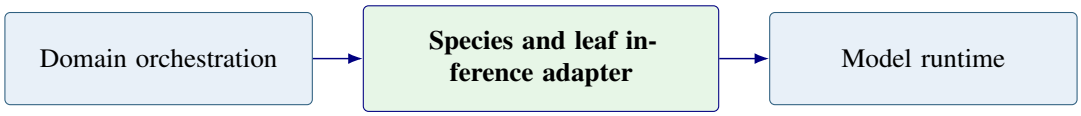

Figure 26: Species inference adapter: hosted or local model runtime behind a stable contract.

7.1.1 ASSUMPTIONS

- Model support depends on available datasets, training quality, and image conditions.
- The system must document model limitations and confidence behavior.

7.1.2 RESPONSIBILITIES

- Accept a plant image or image reference for inference.
- Return labels, confidence values, and any supported feature data.
- Avoid blocking core capture storage when model service is unavailable.

7.1.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
PR-S1	Domain orchestration to inference adapter	Image or image reference	Raw model result
PR-S2	Inference adapter to normalization	Labels and scores	Normalized result request

Table 23: Species and leaf inference adapter interfaces

7.2 TRAY VISION AND LAYOUT ADAPTER SUBSYSTEM

The tray vision adapter supports optional layout-oriented analysis such as plant-region identification, tray structure review, or count estimates. The current rack design prioritizes individual plant captures, so tray-wide inference is treated as optional.

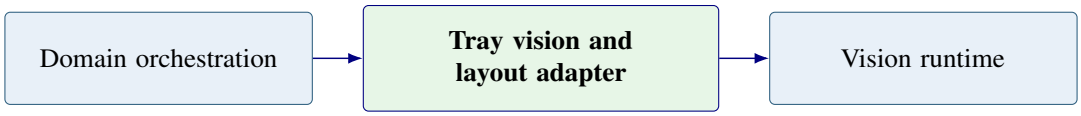

Figure 27: Tray vision adapter: optional tray-level interpretation for rack imagery.

7.2.1 ASSUMPTIONS

- Tray-level results may be less reliable than individual plant captures.
- The adapter must not assume one fixed rack size or camera angle.

7.2.2 RESPONSIBILITIES

- Accept tray or rack images when available.
- Return regions, counts, or layout estimates when configured.
- Degrade gracefully when the model cannot provide a confident result.

7.2.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
PR-T1	Domain orchestration to tray vision adapter	Tray image or metadata	Layout result
PR-T2	Tray vision adapter to normalization	Regions and confidence	Normalized layout result

Table 24: Tray vision and layout adapter interfaces

7.3 INFERENCE NORMALIZATION SUBSYSTEM

Inference normalization converts model-specific output into one canonical shape for persistence and presentation. This allows future models to be swapped without rewriting UI history views.

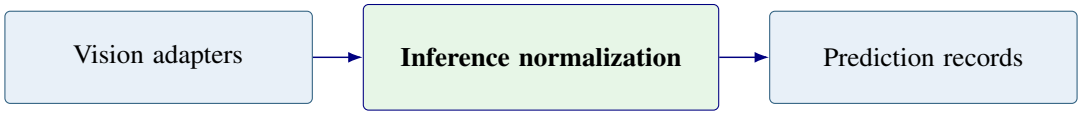

Figure 28: Normalization subsystem: single canonical shape for downstream persistence.

7.3.1 ASSUMPTIONS

- Different models may return different labels, confidence formats, or geometry formats.
- Stored results must remain interpretable even after a model is replaced.

7.3.2 RESPONSIBILITIES

- Normalize labels, confidence values, timestamps, and optional geometry.
- Attach model-source and limitation metadata when available.
- Return a persistence-ready result object.

7.3.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
PR-N1	Vision adapter to normalization	Raw model output	Normalized result
PR-N2	Normalization to domain orchestration	Canonical result	Persistence request

Table 25: Inference normalization subsystem interfaces

7.4 EDGE DISEASE HOOK SUBSYSTEM

The edge disease hook runs after a Raspberry Pi capture is ingested when optional plant-health analysis is configured. It enriches the capture history with structured classification results while keeping raw image storage independent.

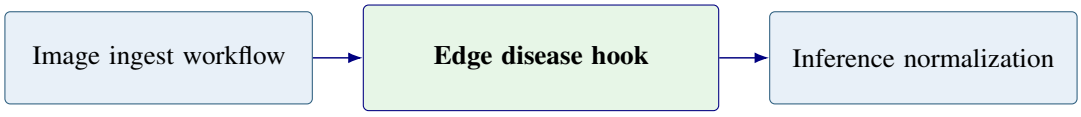

Figure 29: Edge disease hook: optional classification after Pi capture ingest.

7.4.1 ASSUMPTIONS

- A capture must be stored before analysis results are attached.
- Analysis may be disabled for demonstrations or unsupported plant types.

7.4.2 RESPONSIBILITIES

- Trigger optional image analysis after successful capture ingest.
- Associate the result with the correct plant, tray, capture, and model metadata.
- Report failure without losing the original captured image.

7.4.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
PR-D1	Image ingest to disease hook	Capture record and image reference	Analysis request
PR-D2	Disease hook to normalization	Raw classification result	Normalized prediction

Table 26: Edge disease hook subsystem interfaces

7.5 CAPTURE SCHEDULE RUNNER SUBSYSTEM

The capture schedule runner evaluates due capture schedules and enqueues edge-device commands. It allows automatic monitoring to proceed without an operator clicking Take Picture every time.

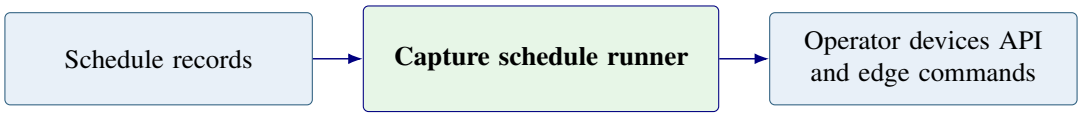

Figure 30: Capture schedule runner: enqueues edge capture work for agents.

7.5.1 ASSUMPTIONS

- Schedules may target trays, plants, or pose sequences.
- The runner must honor disabled schedules and avoid duplicate command creation.

7.5.2 RESPONSIBILITIES

- Evaluate due schedules and target capture poses.
- Create edge capture commands for eligible devices.
- Record scheduling outcomes for operator review.

7.5.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
PR-C1	Schedule records to runner	Due schedule and target data	Capture command request
PR-C2	Runner to edge persistence	Scheduled command	Pending command record

Table 27: Capture schedule runner subsystem interfaces

8 HARDWARE LAYER SUBSYSTEMS

The Hardware Layer is the physical capture plane. It includes Raspberry Pi edge hosts, the rack-mounted XY camera carriage, the camera module, motion components, and the local AgriHome agent that bridges hardware actions to application APIs.

8.1 RASPBERRY PI EDGE HOST SUBSYSTEM

The Raspberry Pi edge host provides local compute for device registration, heartbeat reporting, camera control, motion command execution, and image transfer.

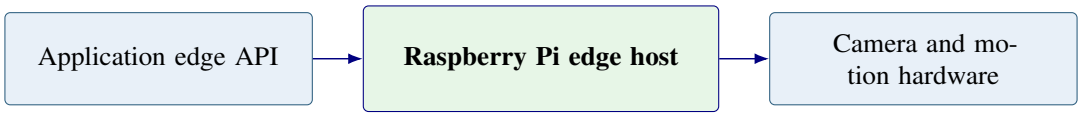

Figure 31: Raspberry Pi edge host subsystem: register, heartbeat, command, capture, and ingest.

8.1.1 ASSUMPTIONS

- The Raspberry Pi has network access, camera access, and safe access to motion controls.
- The device can lose connectivity and later recover without losing identity.

8.1.2 RESPONSIBILITIES

- Maintain device identity and local configuration.
- Run the local AgriHome agent process.
- Coordinate camera and motion commands with status reporting.

8.1.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
H-P1	Pi host to edge API	Register, heartbeat, status, ingest	Commands and acknowledgments
H-P2	Pi host to camera and motion hardware	Capture and movement commands	Image and position status

Table 28: Raspberry Pi edge host subsystem interfaces

8.2 RACK-MOUNTED CAMERA CAPTURE SUBSYSTEM

The rack-mounted camera capture subsystem includes the camera module, XY carriage, rails or fixtures, travel limits, and physical target positions used to capture individual plant images.

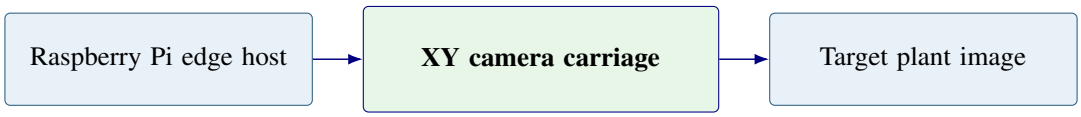

Figure 32: Rack-mounted capture subsystem: camera carriage moves on X and Y axes to target plants.

8.2.1 ASSUMPTIONS

- The camera is attached to the shelf or rack motion system.
- X-axis and Y-axis travel limits prevent unsafe movement.
- Lighting, focus, and plant placement affect image quality.

8.2.2 RESPONSIBILITIES

- Move the camera to defined target positions.
- Capture individual plant images with consistent framing.
- Return image data and movement status to the edge host.

8.2.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
H-C1	Edge host to camera carriage	Target pose and capture command	Movement and capture result
H-C2	Camera carriage to edge host	Image bytes and hardware status	Ingest-ready capture

Table 29: Rack-mounted camera capture subsystem interfaces

8.3 AGRIHOME AGENT BRIDGE SUBSYSTEM

The AgriHome agent bridge runs on the Raspberry Pi and connects application-level commands to local hardware actions. It claims commands, moves the camera carriage, captures images, and uploads results.

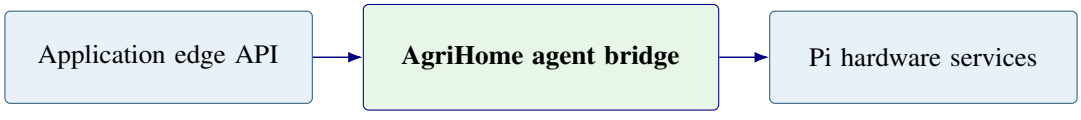

Figure 33: AgriHome agent bridge subsystem: connects edge API commands to local hardware actions.

8.3.1 ASSUMPTIONS

- The agent has access to the configured device key and local hardware interfaces.
- The agent can retry status reporting after temporary network loss.

8.3.2 RESPONSIBILITIES

- Poll or receive pending commands from the Application Layer.
- Execute movement and capture workflows safely.
- Upload captured images, metadata, and command status.

8.3.3 SUBSYSTEM INTERFACES

ID	Interface	Inputs	Outputs
H-A1	Agent to edge API	Heartbeat and command claim	Pending command
H-A2	Agent to hardware services	Move and capture instruction	Image and hardware result

Table 30: AgriHome agent bridge subsystem interfaces

9 REFERENCES

[1] ISO/IEC/IEEE, *ISO/IEC/IEEE 42010:2022 Systems and Software Engineering – Architecture Description*, International Organization for Standardization and IEEE, 2022.

[2] IEEE Computer Society, *IEEE Std 1016-2009: IEEE Standard for Information Technology – Systems Design – Software Design Descriptions*, IEEE, 2009.

[3] C. Bersani, C. Ruggiero, R. Sacile, A. Soussi, and E. Zero, "Internet of things approaches for monitoring and control of smart greenhouses in Industry 4.0," *Energies*, vol. 15, no. 10, p. 3834, 2022.

[4] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, no. 1419, 2016.

[5] J. F. Humplík, D. Lazár, A. Husičková, and L. Spíchal, "Automated phenotyping of plant shoots using imaging methods for analysis of plant stress responses: A review," *Plant Methods*, vol. 11, p. 29, 2015.

[6] R. T. Fielding, M. Nottingham, and J. Reschke, "HTTP Semantics," Internet Engineering Task Force, Tech. Rep. RFC 9110, 2022.

[7] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-207, 2020.